

0073A- Compilation Techniques

Lecturer: **Andrea Corradini**

andrea.corradini@unipi.it

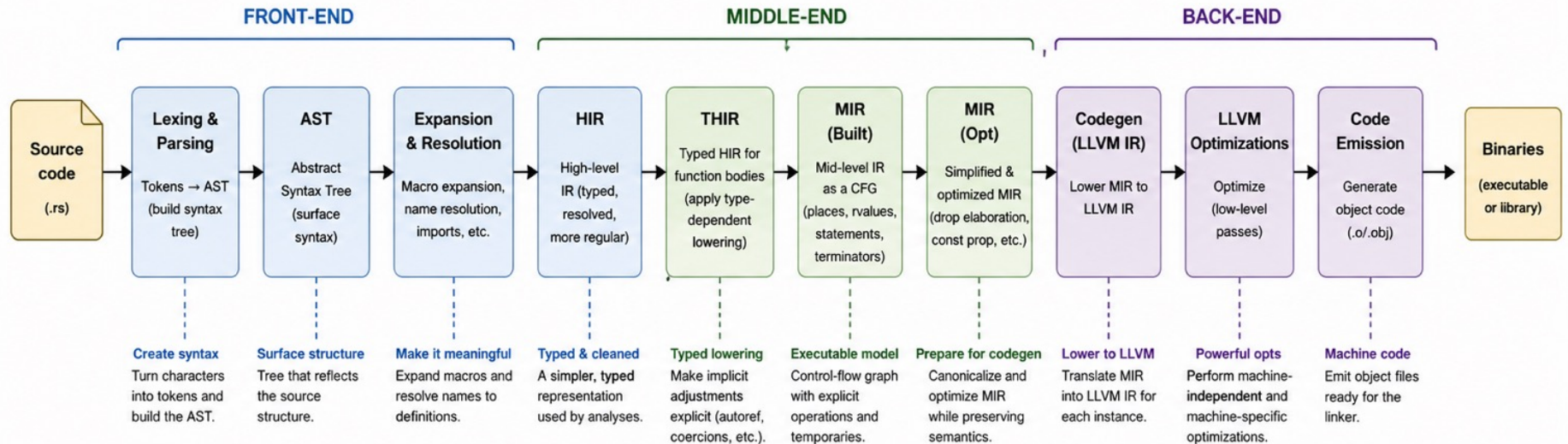
<http://pages.di.unipi.it/corradini/>

CompTech-26: RUST #2

Aspects of Rust Compilation (2)

- How to obtain the various IR's
- Desugaring
- MIR concepts and structure
- MIR Construction through queries
- Visitor pattern and Dataflow analysis on MIR
- Drop elaboration

The rustc Compilation Pipeline



● FRONT-END

- Understand the program text.
- Resolve names and types.
- Produce HIR for the crate.

● MIDDLE-END

- Build and transform MIR.
- Run borrow checking and other analyses.
- Prepare MIR for codegen.

● BACK-END

- Monomorphize generic code.
- Generate LLVM IR.
- Optimize and emit object code.

● AFTER rustc

- Link object files and libraries.
- Produce final binaries.

Key idea

Ownership and borrowing are enforced statically in the middle-end over MIR, well before code reaches the backend.

AST = Abstract Syntax Tree • HIR = High-Level IR • THIR = Typed HIR • MIR = Mid-Level IR • CFG = Control-Flow Graph

Compiler options for IR's dumps

- **HIR dump**: ``cargo rustc -- -Z unpretty=hir-tree`` shows the high-level tree; the ``-Z`` flag requires a nightly compiler.
- **THIR dump**: ``cargo rustc -- -Z unpretty=thir-tree`` exposes typed body lowering and adjustments before MIR construction.
- **MIR dump**: ``rustc -Z dump-mir=all`` emits MIR at several stages, useful for observing construction, analysis, and cleanup.
- **Backend artifacts**: ``rustc --emit=mir`` and ``rustc --emit=llvm-ir`` produce artifacts that can be compared to see what disappears before LLVM.

From MIR to LLVM IR: What Survives?

- **Survives**: Control flow, calls, loads, stores, arithmetic operations, layouts, and concrete monomorphized functions survive into backend-oriented forms.
- **Does not survive directly**: Lifetimes and ownership are not first-class runtime values in the generated code. They serve to validate the program before lowering.
- **Cleanup matters**: Post-borrowck cleanup removes checking-only artifacts and prepares a runtime MIR dialect suitable for transformations and codegen.
- **LLVM boundary**: LLVM receives lower-level IR where Rust-specific safety reasoning has already justified many assumptions.

Desugaring

- The various phases of lowering towards MIR perform various kinds of desugaring
- Early desugaring (e.g., for-loops) happens during HIR lowering.
- Late desugaring (method calls, autoderref, autoref, operators) happens when building THIR.
- MIR construction removes expression nesting entirely.

Syntactic sugar in Rust

- Lot of syntactic sugar in Rust
- It hides important information, also needed to perform borrow-checking
- Examples:
 - Method Call Desugaring
 - for-loop Desugaring
 - Operator Overloading
 - ? operator → match on Result / Option
 - if let → match
 - while let → loop + match
 - auto-ref/deref
 - Closure → struct + trait (Fn, FnMut, FnOnce)
 - Indexing v[i] → Index trait
 - return implicit → tail expression

Examples of desugaring

Source	Desugared
<pre>for x in v { println!("{}", x); }</pre>	<pre>{ let mut iter = IntoIterator::into_iter(v); loop { match iter.next() { Some(x) => { println!("{}", x); } None => break, } } }</pre>
<pre>let x = foo()?;</pre> <pre>enum Result<T, E>{ Ok(T), Err(E), }</pre>	<pre>let x = match foo() { Ok(v) => v, Err(e) => return Err(From::from(e)), };</pre>
<pre>let x = opt?;</pre> <pre>enum Option<T> { None, Some(T), }</pre>	<pre>let x = match opt { Some(v) => v, None => return None, };</pre>
<pre>v.push(10);</pre>	<pre>Vec::push(&mut v, 10);</pre>
<pre>x.foo()</pre>	<pre>Foo::foo(&x) Foo::foo(&mut x) Foo::foo(&*x)</pre>

Method syntax is **just sugar**

Compiler inserts:

- &, &mut
- deref (*)
- trait resolution

Examples of desugaring (2)

Source	Desugared
<pre>if let Some(x) = opt { println!("{}", x); }</pre>	<pre>match opt { Some(x) => { println!("{}", x); } _ => {} }</pre>
<pre>while let Some(x) = iter.next() { println!("{}", x); }</pre>	<pre>loop { match iter.next() { Some(x) => { println!("{}", x); } None => break, } }</pre>
<pre>v[i] // indexing</pre>	<pre>*std::ops::Index::index(&v, i) // Index trait</pre>
<pre>v[i] = 10;</pre>	<pre>*std::ops::IndexMut::index_mut(&mut v, i) = 10;</pre>
<pre>fn f() -> i32 { 3 } // implicit return</pre>	<pre>fn f() -> i32 { return 3; } // tail expression</pre>

Examples of desugaring (3)

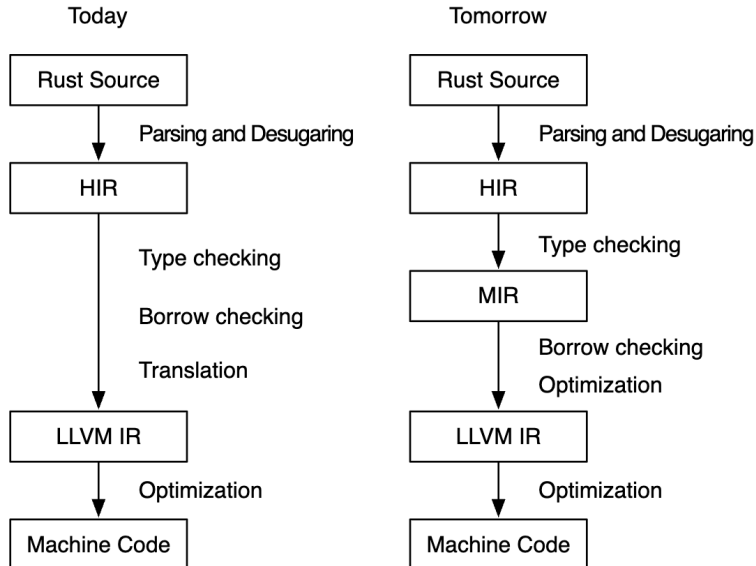
Source	Desugared
<pre>let f = x x + 1; // closure</pre>	<pre>struct Closure; impl Fn(i32) -> i32 for Closure { extern "rust-call" fn call(&self, (x,): (i32,)) -> i32 { x + 1 } } let f = Closure;</pre>
<pre>let y = 10; let f = x x + y; // variable capturing</pre>	<pre>struct Closure { y: i32 } impl Fn(i32) -> i32 for Closure { extern "rust-call" fn call(&self, (x,): (i32,)) -> i32 { x + self.y } } let f = Closure { y };</pre>

rustc internal desugaring:

- The resulting syntax is not legal Rust code
- Capture inference and trait selection is not expressible directly by user-written code

Introducing the MIR

April 2016



Benefits

- Faster compilation time
- Faster execution time
- More precise borrow checking - NLL

MIR: key characteristics and concepts

- It is based on a control-flow graph.
- It does not have nested expressions.
- All types in MIR are fully explicit (built from THIR).
- **Basic blocks:** units of the control-flow graph, consisting of:
 - **statements:** actions with one successor
 - **terminators:** actions with potentially multiple successors; always at the end of a block
- **Locals:** function arguments, local variables, and temporaries.
like `_1`, `_2`, `Local _0` : return value.
- **Places:** expressions that identify a location in memory,
like `_1` or `_1.f`.
- **Rvalues:** expressions that produce a value.
 - **Operands:** the arguments to an rvalue: either be a constant (like 22) or a place (like `_1`).

MIR of a function

```
fn main() {  
    let mut vec = Vec::new();  
    vec.push(1);  
    vec.push(2); }  
}
```

- Locals unnamed, `_0` for return value.
- «debug» user variables
- MIR records whether values are **copied** or **moved**, information needed for ownership reasoning
- The CFG exposes paths that later analyses can traverse with standard dataflow techniques.

```
// optimized MIR  
fn main() -> () {  
    let mut _0: ();  
    let mut _1: std::vec::Vec<i32>;  
    let _2: ();  
    let mut _3: &mut std::vec::Vec<i32>;  
    let _4: ();  
    let mut _5: &mut std::vec::Vec<i32>;  
    scope 1 {  
        debug vec => _1;  
    }  
    bb0: {  
        _1 = Vec::<i32>::new() -> [return: bb1, unwind  
continue];  
    }  
    bb1: {  
        _3 = &mut _1;  
        _2 = Vec::<i32>::push(move _3, const 1_i32) ->  
[return: bb2, unwind: bb5];  
    }  
    bb2: {  
        _5 = &mut _1;  
        _4 = Vec::<i32>::push(move _5, const 2_i32) ->  
[return: bb3, unwind: bb5];  
    }  
    bb3: {  
        drop(_1) -> [return: bb4, unwind continue];  
    }  
    bb4: {  
        return;  
    }  
    bb5 (cleanup): {  
        drop(_1) -> [return: bb6, unwind terminate(cleanup)];  
    }  
    bb6 (cleanup): {  
        resume;  
    }  
}
```

MIR Data Types

- The MIR data types are defined in the [compiler/rustc_middle/src/mir/](#) module. Each of the key concepts mentioned before maps to a Rust type.
- The main MIR data type is [Body](#).
- **Basic blocks**: The basic blocks are stored in the field [Body::basic_blocks](#); this is a vector of [BasicBlockData](#) structures. Nobody ever references a basic block directly: instead, we pass around [BasicBlock](#) values, which are [newtype'd](#) indices into this vector.
- **Statements** are represented by the type [Statement](#).
- **Terminators** are represented by the [Terminator](#).
- **Locals** are represented by a [newtype'd](#) index type [Local](#). The data for a local variable is found in the [Body::local_decls](#) vector. There is also a special constant [RETURN_PLACE](#) identifying the special “local” representing the return value.
- **Places** are identified by the struct [Place](#). There are a few fields:
 - Local variables like `_1`
 - **Projections**, which are fields or other things that “project out” from a base place. These are represented by the [newtype'd](#) type [ProjectionElem](#). So e.g. the place `_1.f` is a projection, with `f` being the “projection element” and `_1` being the base path. `*_1` is also a projection, with the `*` being represented by the [ProjectionElem::Deref](#) element.
- **Rvalues** are represented by the enum [Rvalue](#).
- **Operands** are represented by the enum [Operand](#).

Mir Queries and Passes

To obtain the MIR:

- for a function - use the optimized_mir query (typically used by codegen) or the mir_for_ctfe query (typically used by compile time function evaluation, i.e., *CTFE*);
- for a promoted - you can use the promoted_mir query.

These return the final, optimized MIR.

- For foreign def-ids (of other crates), the MIR is obtained from the other crate's metadata.
- For local def-ids, the query will construct the optimized MIR by requesting a pipeline of upstream queries. Each query will contain a series of passes.

Mir Queries and Passes (2)

To produce the optimized MIR for a given def-id **D**, the **promoted_mir** query goes through several *suites of passes*, each grouped by a query.

Each suite consists of passes which perform *linting*, *analysis*, *transformation* or *optimization*.

Each query represent a useful intermediate point where we can access the MIR dialect for type checking or other purposes:

- **mir_built(D)** – it gives the initial MIR just after it's built;
- **mir_const(D)** – it applies some simple transformation passes to make MIR ready for const qualification;
- **mir_promoted(D)** - it extracts promotable temps into separate MIR bodies, and also makes MIR ready for borrow checking;
- **mir drops elaborated and const checked(D)** - it performs borrow checking, runs major transformation passes (such as **drop elaboration**) and makes MIR ready for optimization;
- **optimized mir(D)** – it performs all enabled optimizations and reaches the final state.

MIR promotion

A **promoted MIR** is a piece of MIR that has been extracted from a function body and lifted to a separate, constant-like MIR body so that it can be **evaluated at compile time** or treated as **having 'static lifetime**.

- During MIR construction the compiler identifies expressions that are **pure, constant, and free of side effects** (e.g., array literals, references to constant data). Instead of keeping them inline in the function's MIR, it **promotes** them into a separate MIR body (a **promoted constant**), and replaces the original expression with a reference to that promoted value.

Example:

```
fn f() -> &'static [i32; 3] {  
    &[1, 2, 3]  
}
```

- Without promotion, the array would be a temporary inside **f**.
- With promotion, the compiler turns it into something like:

```
Promoted[0]:  
_0 = [1, 2, 3];  
return;  
  
fn f():  
_0 = &'static Promoted[0];  
return;
```

MIR Construction

The `mir_built` query causes the lowering from THIR of

- Function and closure bodies
- Initializers of static and const items
- Initializers of enum discriminants

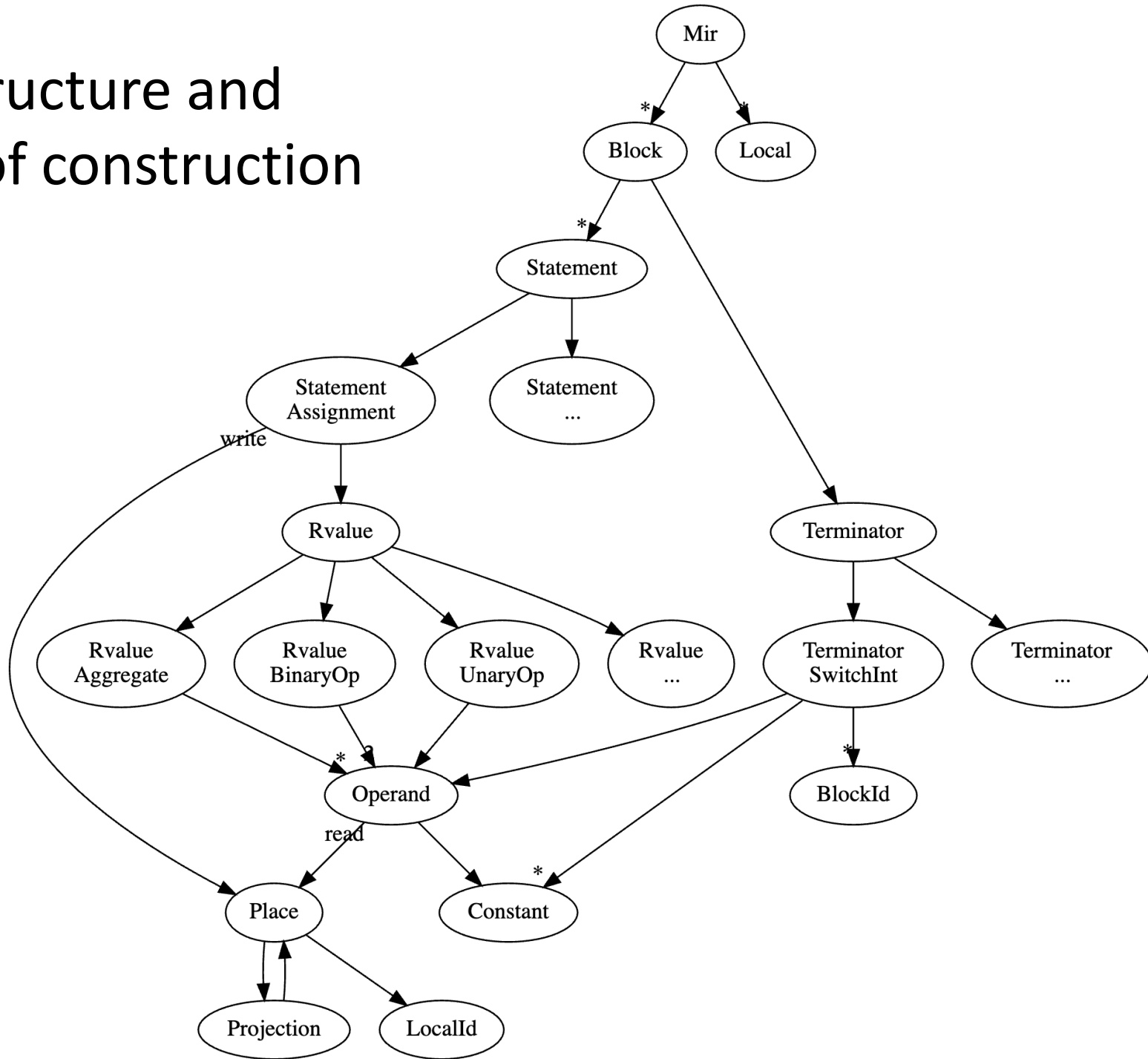
and other things.

It first creates

- local variables for every argument
- local variables for every binding specified
- field accesses that read the fields from the argument, and writes the value to the binding variable

Finally it triggers a recursive call to a function that generates the MIR for the body (a Block expression) and writes the result into the `RETURN_PLACE`.

MIR structure and order of construction



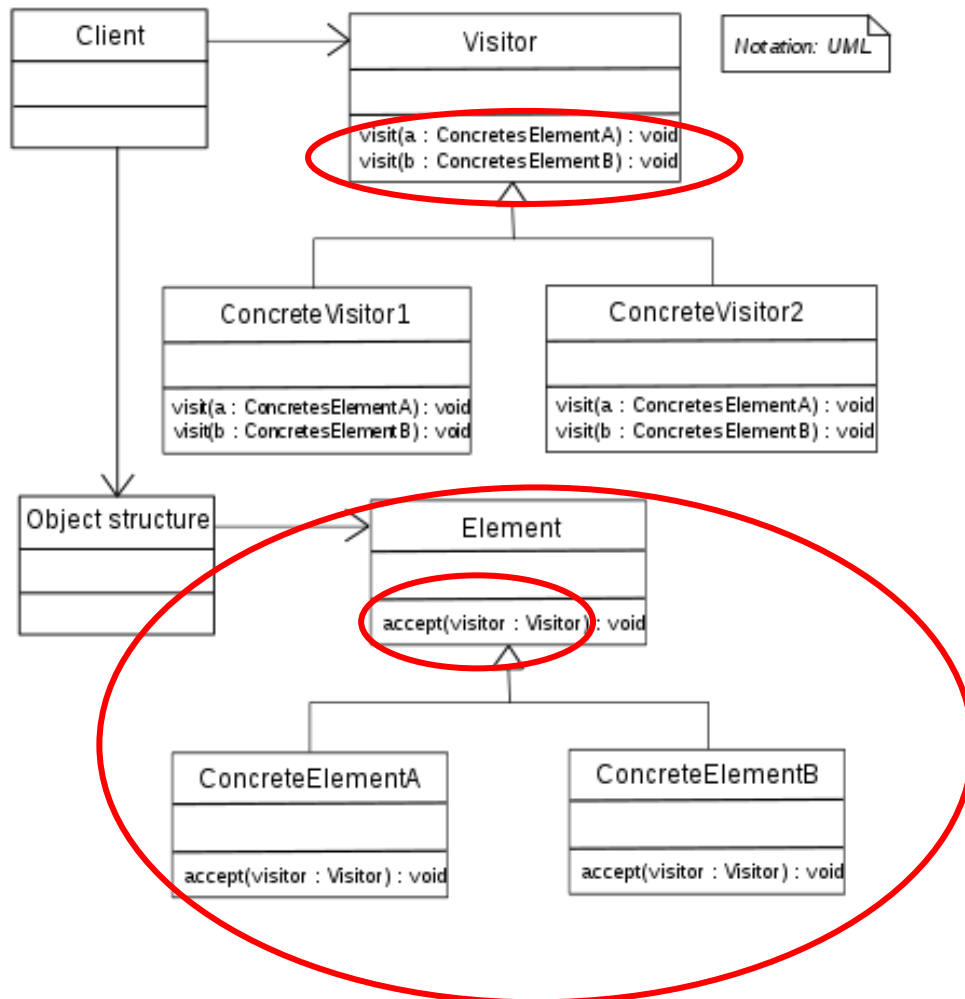
MIR Visitor

The **MIR visitor** is a convenient tool for traversing the MIR and either looking for things or making changes to it. → Used by dataflow analyses

`rustc` supports the visitor design pattern with suitable traits defined in the **middle::mir::visit** module:

- **Visitor** (which operates on a **&Mir** and gives back shared references)
- **MutVisitor** (which operates on a **&mut Mir** and gives back mutable references).

The **Visitor** design pattern



- The data structure can be made of different types of components (ConcreteElements)
- Each component implements an accept(Visitor) method
- The Visitor defines one **visit** method for each type
- The navigation logic is in the Visitor
- At each step, the correct **visit** method is selected by overloading

MIR Visitor (2)

To implement a visitor, you have to create a type that represents your visitor. Typically, this type wants to “hang on” to whatever state you will need while processing MIR:

```
struct MyVisitor<...> { tcx: TyCtxt<'tcx>, ...  
}
```

and you then implement the **Visitor** or **MutVisitor** trait for that type:

```
impl<'tcx> MutVisitor<'tcx> for MyVisitor {  
    fn visit_foo(&mut self, ...) {  
        ...  
        self.super_foo(...); } }  
}
```

In addition the visitor, the rustc **middle::mir::traversal** module contains useful functions for walking the MIR CFG in different standard orders (e.g. pre-order, reverse post-order, and so forth).

MIR Dataflow Analysis

Dataflow Analysis on the MIR is widely used by rustc.

For example:

- to find uninitialized variables,
- determine what variables are live across a generator yield statement,
- compute which Places are borrowed at a given point in the CFG.

A dataflow analysis is defined by the Analysis trait. In addition to the type of the dataflow state, this trait defines the initial value of that state at entry to each block, as well as the direction of the analysis, either forward or backward. The domain of your dataflow analysis must be a lattice (strictly speaking a join-semilattice) with a well-behaved join operator. See documentation for the lattice module, as well as the JoinSemiLattice trait, for more information.

Transfer Functions and Effects

The dataflow framework in rustc allows each statement (and terminator) inside a basic block to define its own transfer function.

- Transfer functions are called **effects**.
- Each effect is applied successively in dataflow order, and together they define the transfer function for the entire basic block.
- It's also possible to define an effect for particular outgoing edges of some terminators

Statements can also be equipped with **“Before” Effects**

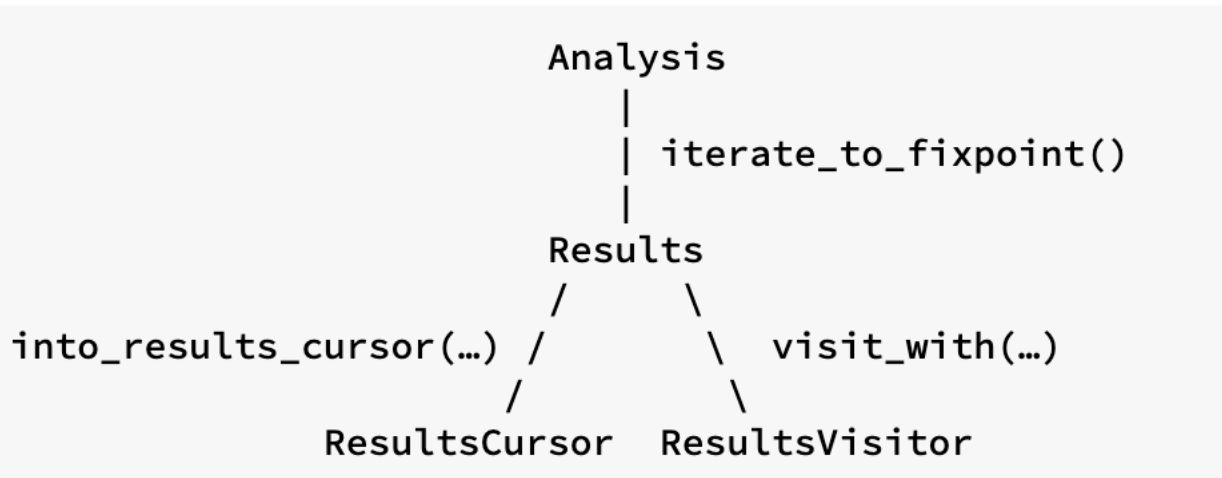
- The **before effects** are applied immediately before the unprefix effect **regardless of the direction of the analysis**. That is, a backward analysis will apply the “before” effect and then the “primary” effect when computing the transfer function for a basic block, just like a forward analysis.
- The “before” variants can be useful in some scenarios, such as when the effect of the right-hand side of an assignment statement must be considered separately from the left-hand side.

Inspecting the Results of a Dataflow Analysis

After constructing an analysis, you must call `iterate_to_fixpoint` which will return a `Results`, which contains the dataflow state at fixpoint upon entry of each block.

Once you have a `Results`, you can inspect the dataflow state at fixpoint at any point in the CFG.

If you only need the state at a few locations (e.g., each Drop terminator) use a `ResultsCursor`. If you need the state at *every* location, a `ResultsVisitor` will be more efficient.



Inspecting the Results (2)

For example, the following code uses a [ResultsVisitor](#)...

```
// Assuming `MyVisitor` implements `ResultsVisitor<FlowState = MyAnalysis::Domain>`...
let mut my_visitor = MyVisitor::new();
// inspect the fixpoint state for every location within every block in RP0.
let results = MyAnalysis::new()
    .iterate_to_fixpoint(tcx, body, None);
results.visit_with(body, &mut my_visitor);
```

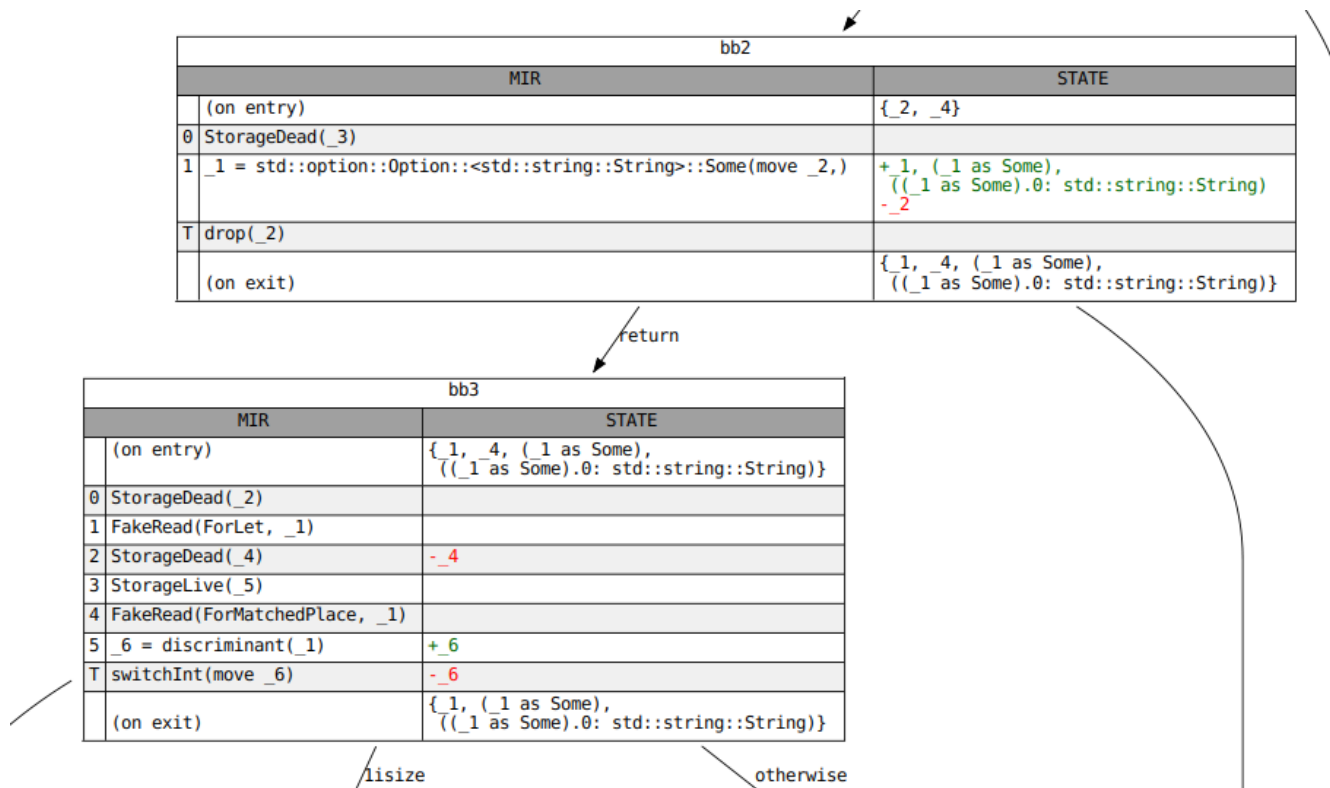
- whereas this code uses [ResultsCursor](#):

```
let mut results = MyAnalysis::new()
    .iterate_to_fixpoint(tcx, body, None);
    .into_results_cursor(body);

// Inspect the fixpoint state immediately before each `Drop`
terminator.
for (bb, block) in body.basic_blocks().iter_enumerated() {
    if let TerminatorKind::Drop { .. } = block.terminator().kind {
        results.seek_before_primary_effect(body.terminator_loc(bb));
        let state = results.get();
        println!("state before drop: {:#?}", state);
    }
}
```

Graphviz Diagrams to inspect dataflow

Each visualization will display the full dataflow state at entry and exit of each block



Drop Elaboration: Dynamic drops

- When an initialized variable or temporary goes out of scope, its destructor is run, or it is **dropped**. Assignment also runs the destructor of its left-hand operand, if it's initialized. If a variable has been partially initialized, only its initialized fields are dropped.
- Whether a variable is initialized or not is only known at runtime

```
let mut y = vec![];
{
    let x = vec![1, 2, 3];
    if std::process::id() % 2 == 0 {
        y = x; // conditionally move `x` into `y`
    }
} // `x` goes out of scope here. Should it be dropped?
```

- When building the MIR, the Drop and DropAndReplace terminators represent places where drops may occur. However, in this phase, the presence of these terminators does not guarantee that a destructor will run.
- In these cases, we need to keep track of whether a variable is initialized *dynamically*.

Drop obligations

- When a local variable becomes initialized, it establishes a set of “drop obligations”:
a set of structural paths (e.g. a local a, or a path to a field b.f.y) that need to be dropped.
- The drop obligations for a local variable x of struct-type T are computed from analyzing the structure of T. If T itself implements Drop, then x is a drop obligation.
If T does not implement Drop, then the set of drop obligations is the union of the drop obligations of the fields of T.
- When a structural path is moved from (and thus becomes uninitialized), any drop obligations for that path or its descendants (path.f, path.f.g.h, etc.) are released.
- When a local variable goes out of scope (Drop), or when a structural path is overwritten via assignment (DropAndReplace), we check for any drop obligations for that variable or path.
- Unless that obligation has been released by this point, its associated Drop implementation will be called. For enum types, only fields corresponding to the “active” variant need to be dropped. When processing drop obligations for such types, we first check the discriminant to determine the active variant. All drop obligations for variants besides the active one are ignored.

Drop elaboration

- One valid model for these rules is to keep a boolean flag (a “drop flag”) for every structural path that is used at any point in the function.
- This flag is set when its path is initialized and is cleared when the path is moved from. When a Drop occurs, we check the flags for every obligation associated with the target of the Drop and call the associated Drop impl for those that are still applicable.
- This process—transforming the newly built MIR with its imprecise Drop and DropAndReplace terminators into one with drop flags—is known as **drop elaboration**.
- When a MIR statement causes a variable to become initialized (or uninitialized), drop elaboration inserts code that sets (or clears) the drop flag for that variable. It wraps Drop terminators in conditionals that check the newly inserted drop flags.
- Once this is complete, Drop terminators in the MIR correspond to a call to the “drop glue” or “drop shim” for the type of the dropped place. The drop glue for a type calls the Drop impl for that type (if one exists), and then recursively calls the drop glue for all fields of that type.

Drop elaboration in rustc

A few optimizations:

- Only paths that are the target of a Drop (or have the target as a prefix) need drop flags.
- Some variables are known to be initialized (or uninitialized) when they are dropped. These do not need drop flags.
- If a set of paths are only dropped or moved from via a shared prefix, those paths can share a single drop flag.

A subset of these are implemented in rustc.

Drop elaboration designates each Drop in the newly built MIR as one of four kinds:

- **Static**, the target is always initialized.
- **Dead**, the target is always **un**initialized.
- **Conditional**, the target is either wholly initialized or wholly uninitialized. It is not partly initialized.
- **Open**, the target may be partly initialized.

For this, it uses a pair of dataflow analyses, **MaybeInitializedPlaces** and **MaybeUninitializedPlaces**.

If a place is in one but not the other, then it is known at compile-time if it is **Dead** or **Static**: no need for a flag for the target. The Drop terminator is removed (**Dead**) or preserved (**Static**).

For Conditional drops, a drop flag is generated for the target.

Open Drops

Concretely, in MIR construction and early analysis phases, a drop(x) may be “open” in the sense that:

- The compiler knows that x must be dropped,
- but it has not yet fully elaborated **how** that drop happens.

This includes:

- whether x has a custom Drop implementation,
- whether x is partially initialized or partially moved.
- which fields of x still need to be dropped,
- and in which order nested drops must occur.

Later in the pipeline, during **drop elaboration**, these open drops are transformed into **fully explicit sequences of drop operations** (sometimes called “closed” drops), where:

- all control-flow paths are made explicit,
- all field-wise drops are inserted,
- and conditional drops (depending on moves) are handled precisely.

Why it exists

This separation allows the compiler to:

- keep MIR simpler during early analyses (borrow checking, dataflow),
- and only later expand drops into their full low-level form needed for correct execution.

So, an Open Drop is essentially a **placeholder for destruction semantics** that gets refined into concrete operations in later MIR passes.